

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №6
по курсу «Разработка кроссплатформенных приложений»
на тему «Сетевое взаимодействие в Java»

Вариант 7

Выполнили студенты группы 23ВВИ1:

Макаров А. С.

Орищенко М. Д.

Приняла:

к. т. н., доцент Юрова О. В.

Пенза 2026

Содержание

- 1 Цель работы
 - 2 Задание на лабораторную работу
 - 3 Краткие теоретические сведения
 - 4 Описание разработанного приложения
 - 5 Алгоритм работы программы
 - 6 Ход выполнения работы
 - 7 Тестирование программы
 - 8 Какие скриншоты нужно вставить в отчет
 - 9 Вывод
- Приложение А. Основные фрагменты исходного кода

1 Цель работы

Цель работы: научиться создавать клиент-серверные приложения с использованием стандартных классов Java, а также закрепить навыки распределенного вычисления задачи на нескольких вычислительных узлах.

2 Задание на лабораторную работу

Модифицировать приложение из предыдущей лабораторной работы, реализовав клиент-серверную архитектуру для распределенного вычисления определенного интеграла на нескольких вычислительных узлах. Каждый вычислительный узел должен использовать несколько нитей, как в предыдущей лабораторной работе.

Серверная часть не выполняет вычисления. Она отвечает за взаимодействие с пользователем, отправку задач клиентам и агрегацию полученных результатов. Для нечетных вариантов используется протокол UDP, поэтому для варианта 7 обмен данными реализован через UDP-дейтаграммы.

В разработанной программе вычисляется определенный интеграл функции $f(x) = 1 / \ln(x)$ методом трапеций. Пользователь вводит нижний предел, верхний предел и шаг вычисления в графическом интерфейсе Swing.

3 Краткие теоретические сведения

Java поддерживает сетевое взаимодействие с помощью стандартных классов пакета `java.net`. В данной работе используются классы `DatagramSocket` и `DatagramPacket`, так как вариант лабораторной работы требует применения протокола UDP.

UDP не устанавливает постоянного логического соединения между узлами. Данные передаются отдельными пакетами - дейтаграммами. Такой способ обычно проще и быстрее, но он не гарантирует доставку, порядок получения и отсутствие потерь. Поэтому на стороне сервера необходимо связывать ответ с исходной задачей с помощью идентификатора.

Для передачи объекта задачи между сервером и клиентами используется сериализация. Класс `RecIntegral` реализует интерфейс `Serializable`, поэтому объект с пределами интегрирования, шагом и результатом может быть записан в поток `ObjectOutputStream` и восстановлен через `ObjectInputStream`.

Для параллельного вычисления используется многопоточность. Клиент получает часть общего интервала, делит ее на несколько подынтервалов и запускает нити CalculationThread. После завершения нитей частичные суммы складываются и отправляются серверу.

4 Описание разработанного приложения

Приложение состоит из серверной части с графическим интерфейсом и нескольких вычислительных клиентов. Сервер реализован в классе MainFrame, а запуск графического окна выполняется из класса JavaLab1. Клиентская часть реализована в классе ComputeClient.

В программе используются три клиента. Сервер отправляет им части выбранного интервала на порты 12345, 12347 и 12348. Ответы от клиентов сервер принимает на порту 12346. Каждый клиент после получения задачи делит ее еще на семь частей и выполняет вычисление в семи нитях.

Файл	Назначение
JavaLab1.java	Запускает графическое окно MainFrame в потоке обработки событий Swing.
MainFrame.java	Серверная часть: GUI, таблица данных, отправка задач клиентам, прием и агрегация ответов.
ComputeClient.java	Клиентская часть: ожидание UDP-задач, параллельное вычисление и отправка результата серверу.
CalculationThread.java	Нить вычисления части интеграла методом трапеций.
RecIntegral.java	Сериализуемый класс задачи: нижний предел, верхний предел, шаг, результат и идентификатор.
InvalidException.java	Пользовательское исключение для обработки неверного диапазона значений.

Таблица 1 - Назначение основных классов программы

Узел	Порт	Назначение
ComputeClient 1	12345	Принимает первую часть задачи от сервера.
ComputeClient 2	12347	Принимает вторую часть задачи от сервера.
ComputeClient 3	12348	Принимает третью часть задачи от сервера.
Сервер MainFrame	12346	Принимает результаты вычислений от клиентов.

Таблица 2 - Используемые порты UDP

5 Алгоритм работы программы

1. Пользователь запускает серверную часть JavaLab1, после чего открывается окно MainFrame.
2. Отдельно запускаются три экземпляра ComputeClient с портами 12345, 12347 и 12348.
3. Пользователь вводит нижний предел, верхний предел и шаг вычисления, затем нажимает кнопку Add.
4. После выбора строки и нажатия Calculate сервер делит общий интервал на три части по количеству клиентов.
5. Каждая часть интервала упаковывается в объект RecIntegral, сериализуется и отправляется UDP-пакетом соответствующему клиенту.
6. Клиент получает объект задачи, делит свой интервал на семь частей и запускает семь потоков CalculationThread.
7. Каждый поток вычисляет свою частичную сумму методом трапеций.
8. Клиент суммирует частичные результаты потоков, записывает итог в объект RecIntegral и отправляет его серверу.
9. Сервер принимает ответы в отдельном фоновом потоке, складывает частичные суммы клиентов и обновляет поле Result в таблице JTable.

Схема обмена данными между компонентами программы:

```

Пользователь -> MainFrame (сервер GUI)
MainFrame -> ComputeClient 1: часть интервала [a; a1]
MainFrame -> ComputeClient 2: часть интервала [a1; a2]
MainFrame -> ComputeClient 3: часть интервала [a2; b]
ComputeClient -> 7 потоков CalculationThread
CalculationThread -> частичные суммы

```

ComputeClient -> MainFrame: результат своей части
MainFrame -> JTable: общий результат

6 Ход выполнения работы

6.1 Создание клиентского файла

В проект был добавлен файл ComputeClient.java. Он реализует вычислительный узел, который ожидает UDP-пакеты с задачами, выполняет расчет и отправляет результат обратно серверу.

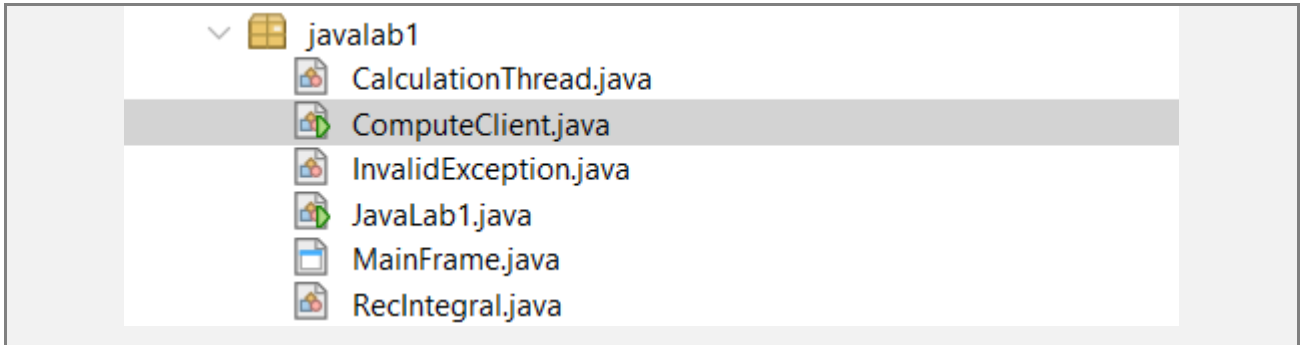


Рисунок 1 - Структура проекта с созданным файлом ComputeClient.java

6.2 Реализация серверной части

Серверная часть расположена в классе MainFrame. При создании окна запускается фоновый поток startResultListener, который слушает порт 12346 и принимает результаты от клиентов. Благодаря отдельному потоку графический интерфейс не зависит во время ожидания UDP-ответов.

```
private void startResultListener() {
    new Thread(() -> {
        try {
            serverSocket = new DatagramSocket(12346);
            System.out.println("Server listening on port 12346...");
            while (true) {
                byte[] buffer = new byte[4096];
                DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
                serverSocket.receive(packet);
                ObjectInputStream ois = new ObjectInputStream(
                    new ByteArrayInputStream(packet.getData())
                );
                RecIntegral resultRec = (RecIntegral) ois.readObject();
                javax.swing.SwingUtilities.invokeLater(() -> {
                    aggregateResult(resultRec);
                });
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }).start();
}
```

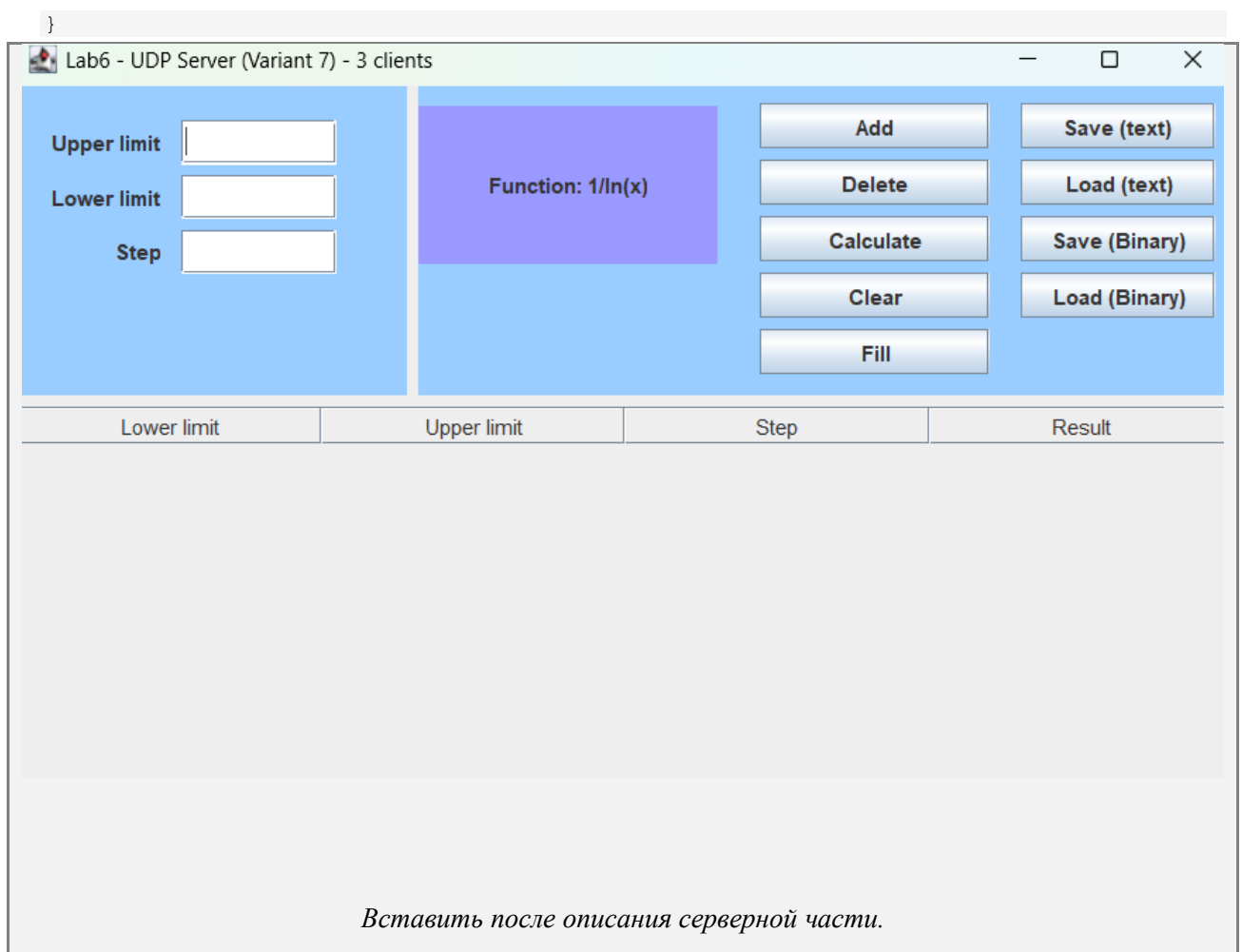


Рисунок 2 - Запущенное окно серверной части программы

6.3 Распределение задачи между клиентами

При нажатии кнопки Calculate сервер проверяет выбранную строку таблицы, делит интервал интегрирования на три части и отправляет каждую часть отдельному клиенту. Для связи результата с исходной строкой используется идентификатор задачи.

```
int clientCount = clientAddresses.length;
double fullInterval = up - low;
double partSize = fullInterval / clientCount;

pendingResults.put(selectedRow, clientCount);
partialSums.put(selectedRow, 0.0);

for (int c = 0; c < clientCount; c++) {
    double partLow = low + c * partSize;
    double partUp = (c == clientCount - 1) ? up : partLow + partSize;
    RecIntegral task = new RecIntegral(partLow, partUp, step);
    task.setId(selectedRow);
    sendTaskToClient(task, clientAddresses[c], clientPorts[c]);
}
```

6.4 Реализация клиентской части

Клиент `ComputeClient` запускается на заданном порту. Порт можно передать через аргумент командной строки или ввести при запуске. После получения объекта `RecIntegral` клиент вызывает метод `computeParallel`, где расчет выполняется в семи потоках.

```
private static double computeParallel(RecIntegral task) {
    double low = task.getLowerBound();
    double up = task.getUpperBound();
    double step = task.getStep();

    double fullInterval = up - low;
    double delta = fullInterval / THREAD_COUNT;

    Thread[] threads = new Thread[THREAD_COUNT];
    double[] partialResults = new double[THREAD_COUNT];

    for (int i = 0; i < THREAD_COUNT; i++) {
        double start = low + i * delta;
        double end = (i == THREAD_COUNT - 1) ? up : start + delta;
        threads[i] = new Thread(
            new CalculationThread(start, end, step, i, partialResults)
        );
        threads[i].start();
    }
    for (Thread t : threads) {
        t.join();
    }

    double totalSum = 0;
    for (double res : partialResults) {
        totalSum += res;
    }
    return totalSum;
}
```

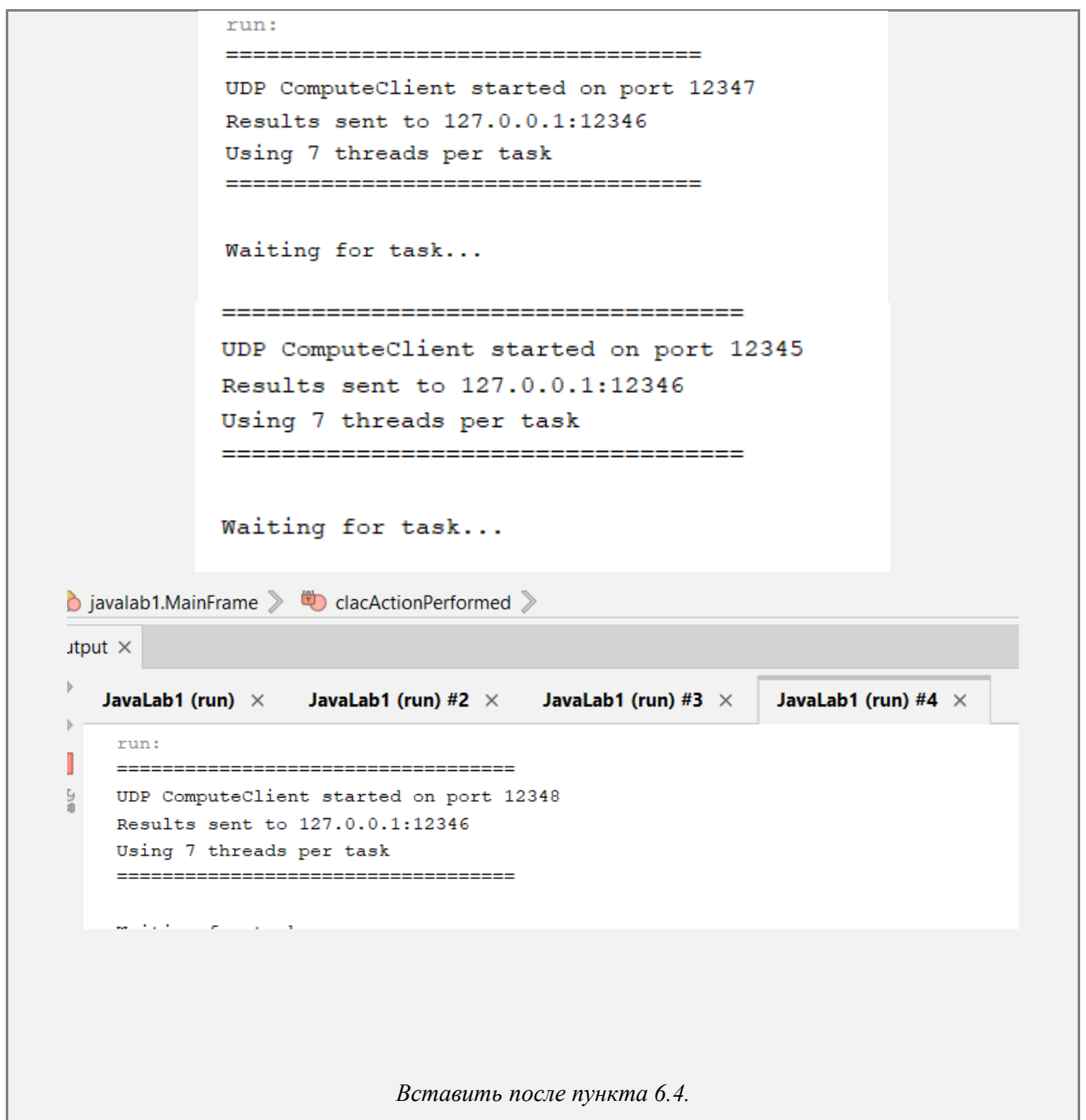



Рисунок 3 - Запуск трех вычислительных клиентов

6.5 Вычисление интеграла в потоках

Каждый поток CalculationThread получает собственный подынтервал и вычисляет площадь методом трапеций. Частичная сумма записывается в массив partialResults по индексу потока.

```
double fPrev = 1.0 / Math.log(xPrev);

for (double x = start + step; x <= end; x += step) {
    if (x <= 0 || x == 1) return;
```

```

        double fCurr = 1.0 / Math.log(x);
        sum += (fPrev + fCurr) / 2 * step;
        fPrev = fCurr;
        xPrev = x;
    }

    partialResults[threadIndex] = sum;

```

6.6 Агрегация результатов на сервере

После получения ответа сервер определяет строку таблицы по идентификатору задачи, добавляет частичную сумму к уже полученным результатам и уменьшает счетчик ожидаемых ответов. Когда ответы пришли от всех клиентов, итоговая сумма записывается в колонку Result.

```

double currentSum = partialSums.getOrDefault(rowIndex, 0.0);
currentSum += Double.parseDouble(resultRec.getResult());
partialSums.put(rowIndex, currentSum);

int remaining = pendingResults.getOrDefault(rowIndex, 0) - 1;
pendingResults.put(rowIndex, remaining);

if (remaining <= 0) {
    String finalResult = String.format("%.6f", currentSum).replace(",", ".");
    model.setValueAt(finalResult, rowIndex, 3);
}

```

6.7 Сохранение и загрузка данных

В программе сохранены функции работы с файлами. Данные таблицы можно сохранить в текстовый файл или бинарный файл, а затем загрузить обратно. Для бинарного сохранения применяется сериализация списка объектов RecIntegral.

7 Тестирование программы

Для проверки работы программы сервер и три клиента запускаются локально на адресе 127.0.0.1. После запуска клиентов пользователь вводит данные в интерфейсе сервера и нажимает Calculate.

Lab6 - UDP Server (Variant 7) - 3 clients

Upper limit
Lower limit
Step

Function: 1/ln(x)

Add
Delete
Calculate
Clear
Fill

Save (text)
Load (text)
Save (Binary)
Load (Binary)

Lower limit	Upper limit	Step	Result
2.0	10000.0	0.01	

Вставить в начале раздела тестирования.

Рисунок 4 - Таблица с добавленными исходными данными до расчета

№	Нижний предел	Верхний предел	Шаг	Ожидаемый результат в таблице
1	2	10	0.1	Около 5.121286
2	2	20	0.1	Около 8.860998
3	10	100	1	Около 23.962073

Таблица 3 - Примеры тестовых данных

Для корректной проверки рекомендуется использовать интервалы, не проходящие через точку $x = 1$, так как функция $1 / \ln(x)$ в этой точке не определена.

```

run:
Server listening on port 12346...
Sending task row 0 to 3 clients
  Client 0: [2.0, 3334.6666666666665], ID=0
  Client 1: [3334.6666666666665, 6667.333333333333], ID=0
  Client 2: [6667.333333333333, 10000.0], ID=0
Received result: 392.373213 for task ID=0
Received result: 483.235637 for task ID=0
Received result: 369.483211 for task ID=0
Row 0 completed: 1245.092061

```

Вставить после таблицы тестовых данных.

Рисунок 5 - Консольные сообщения сервера и клиентов во время обмена UDP-пакетами

Lower limit	Upper limit	Step	Result
2.0	10000.0	0.01	1245.092061

Вставить после выполнения расчета.

Рисунок 6 - Результат вычисления в таблице GUI

8 Какие скриншоты нужно вставить в отчет

Ниже приведен общий список скриншотов. Основные скриншоты 1-6 желательно вставить обязательно. Скриншоты 7-8 можно добавить, если нужно показать работу с файлами и обработку ошибок.

№	Где вставить	Что должно быть на скриншоте
1	Раздел 6.1, после описания создания клиента	Структура проекта в IDE с файлами CalculationThread.java, ComputeClient.java, InvalidException.java, JavaLab1.java, MainFrame.java, RecIntegral.java.
2	Раздел 6.2, после описания сервера	Главное окно Swing-программы: поля ввода, таблица, кнопки управления, подпись Function: $1/\ln(x)$.
3	Раздел 6.4, после описания клиента	Три запущенных клиента ComputeClient на портах 12345, 12347 и 12348, каждый ожидает задачу.
4	Раздел 7, перед расчетом	GUI с добавленной строкой: нижний предел, верхний предел, шаг, пустое поле результата.
5	Раздел 7, во время расчета	Консоль сервера и клиентов: отправка задачи, получение задачи, время расчета, отправка результата.
6	Раздел 7, после расчета	GUI с заполненной колонкой Result после агрегации ответов от клиентов.
7	Раздел 7, после проверки файлов	Сохранение или загрузка данных из .txt/.bin файла. Можно показать окно выбора файла или уже загруженную таблицу.
8	Раздел 7, дополнительная проверка	Сообщение об ошибке при неправильном вводе числа, отрицательном шаге или недопустимом диапазоне.

Таблица 4 - Список скриншотов для отчета

9 Вывод

В ходе лабораторной работы было разработано клиент-серверное приложение на Java с использованием протокола UDP. Серверная часть реализует графический интерфейс, распределяет вычислительную задачу между клиентами и агрегирует результаты. Клиентская часть принимает задачу, выполняет параллельное вычисление интеграла в нескольких потоках и отправляет результат серверу.

В результате были закреплены навыки работы с классами DatagramSocket, DatagramPacket, ObjectInputStream и ObjectOutputStream, а также навыки применения сериализации объектов и многопоточности в Java. Программа может быть масштабирована за счет добавления новых вычислительных узлов.

Приложение А. Основные фрагменты исходного кода

В приложении приведены ключевые фрагменты кода, которые демонстрируют сетевое взаимодействие, распределение задачи и многопоточное вычисление.

А.1 Константы клиента `ComputeClient`

```
private static final int DEFAULT_LISTEN_PORT = 12345;
private static final int SERVER_PORT = 12346;
private static final String SERVER_ADDRESS = "127.0.0.1";
private static final int THREAD_COUNT = 7;
```

А.2 Адреса и порты клиентов в `MainFrame`

```
private final String[] clientAddresses = {"127.0.0.1", "127.0.0.1", "127.0.0.1"};
private final int[] clientPorts = {12345, 12347, 12348};
private final Map<Integer, Integer> pendingResults = new HashMap<>();
private final Map<Integer, Double> partialSums = new HashMap<>();
private final Map<Integer, Integer> taskIdToRow = new HashMap<>();
```

А.3 Отправка результата клиентом

```
RecIntegral resultRec = new RecIntegral(
    task.getLowerBound(),
    task.getUpperBound(),
    task.getStep()
);
resultRec.setId(task.getId());
resultRec.setResult(String.format("%.6f", result).replace(",", "."));
sendResult(resultRec, SERVER_ADDRESS, SERVER_PORT);
```

А.4 Класс `RecIntegral` как сериализуемый объект

```
public class RecIntegral implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private double lowerBound;
    private double upperBound;
    private double step;
    private String result;
}
```